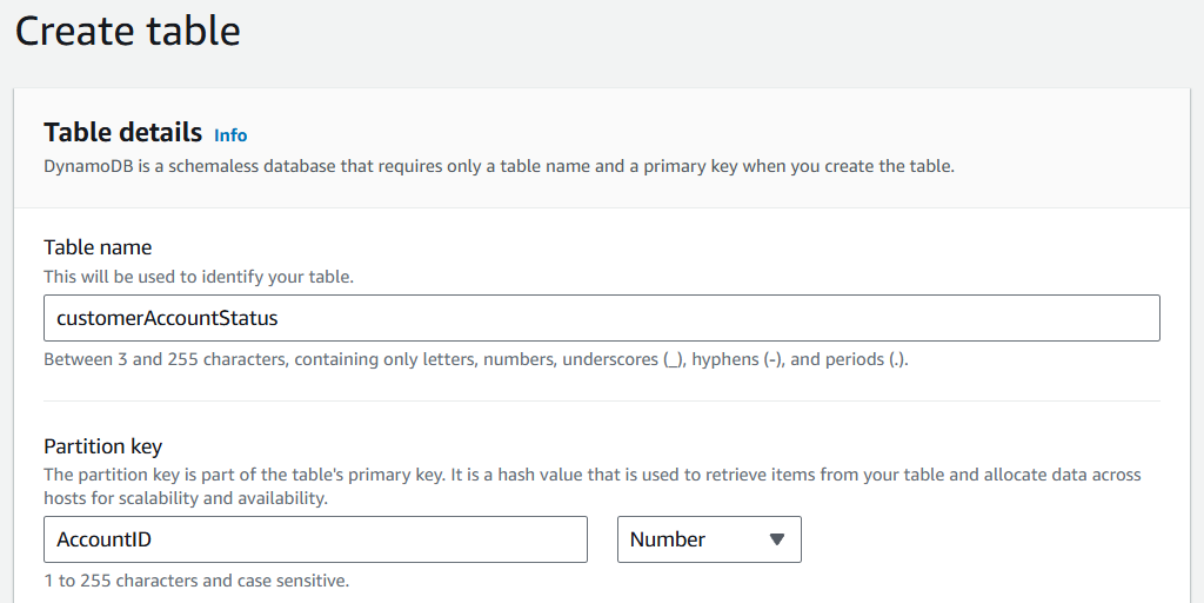
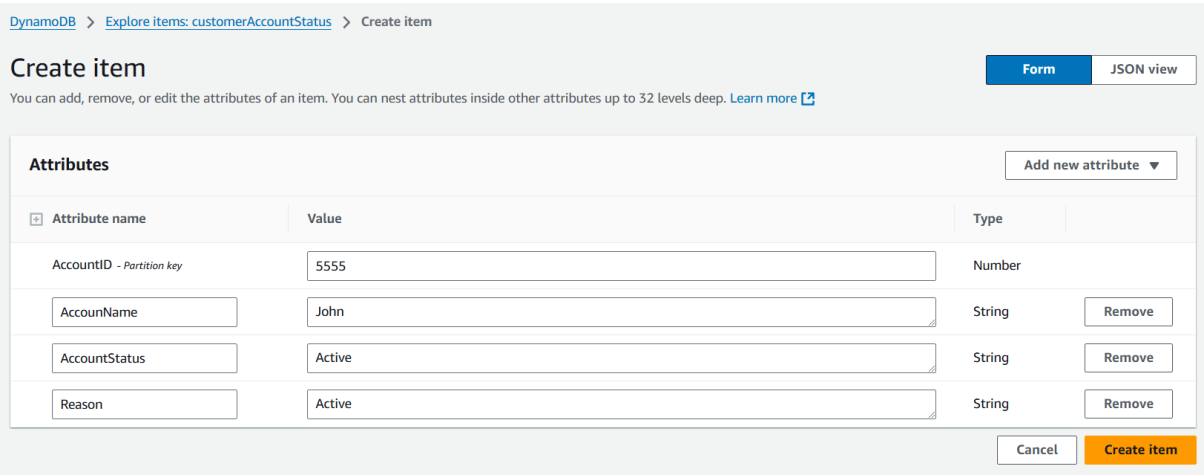
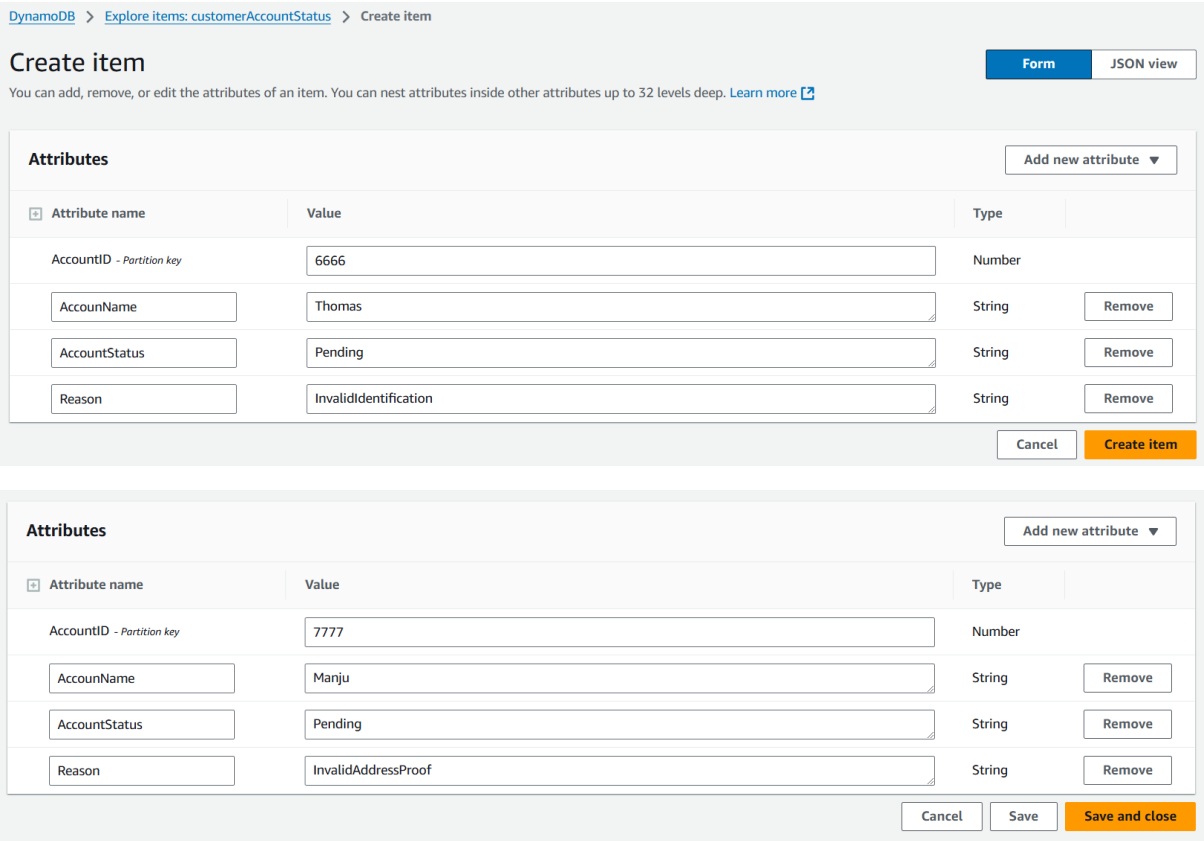
😊 AWS Bedrock (Building a Retail Bank Agent)

1. In this lab, we are going to build a **Retail banking agent** using **Bedrock agents** and **Knowledge base.**
2. Navigate to Dynamo DB and there we will create a table. In the table name give the same name and then in the partition key write Account ID and choose number then click to create this table.

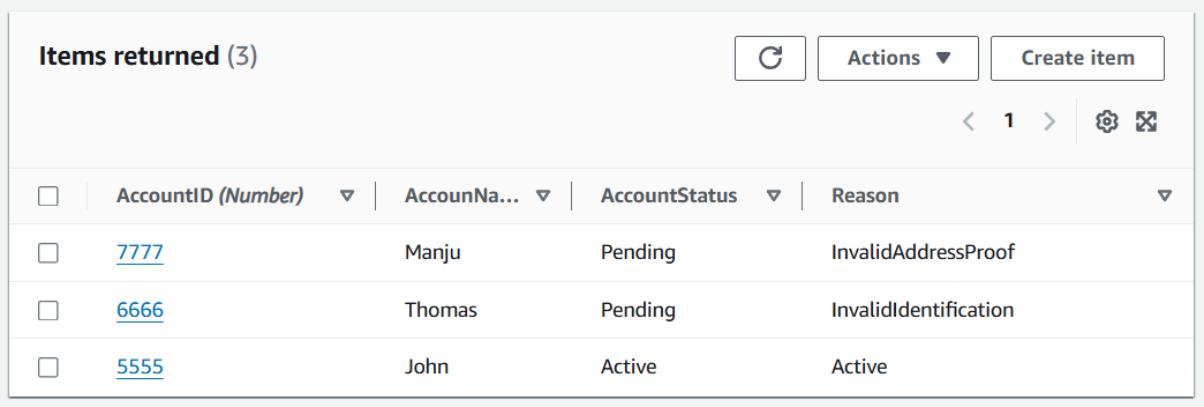


3. Once your table is created, you need to go inside it, click on **Explore Items**, scroll down, and click on **Create Items**.
4. You need to create the same items in the same way as you can see below.

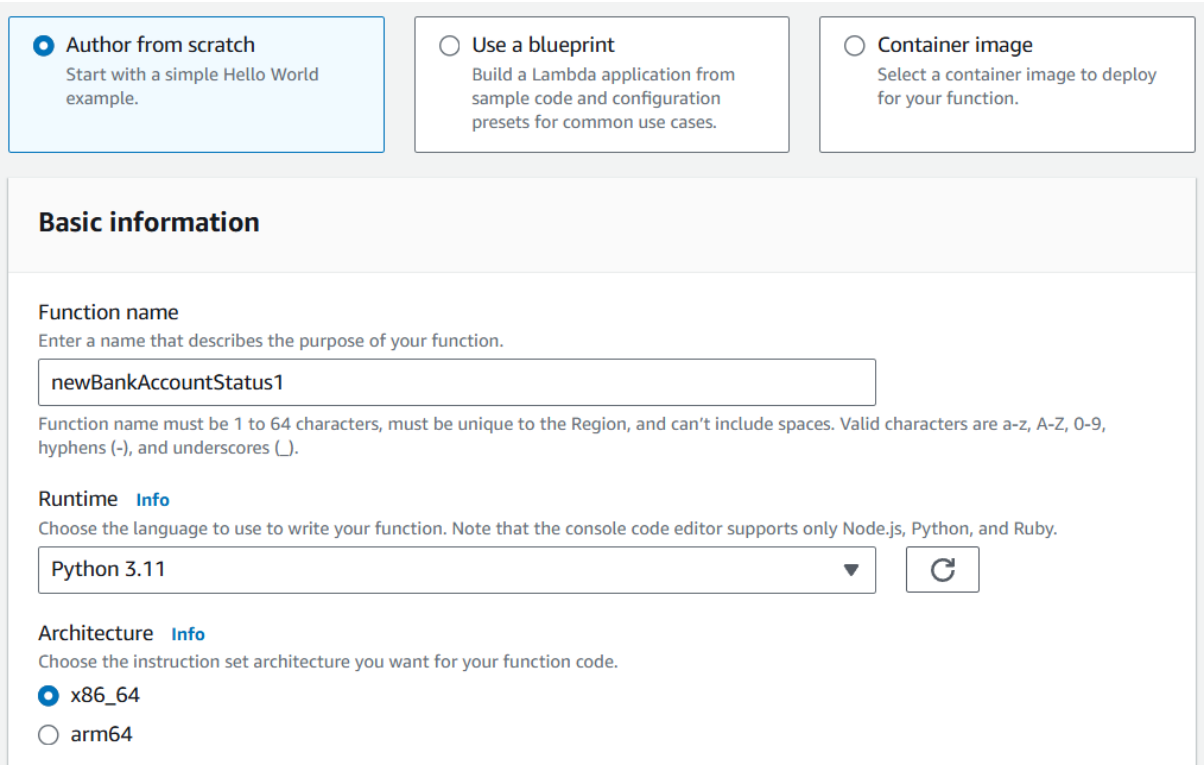




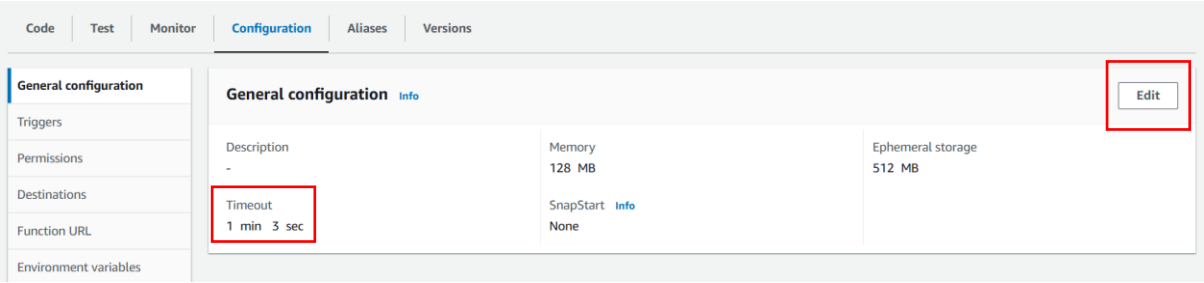
5. Below you can see that we have created all three items that we needed.



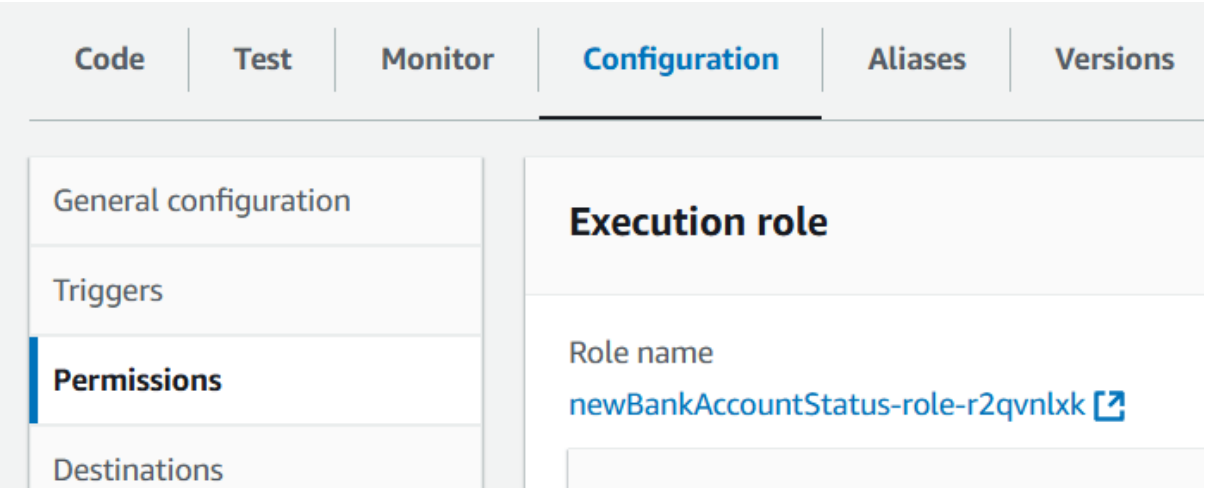
6. Now we need to create a lambda function. So, navigate to it and click on Create. Give it a name and choose Python as your runtime environment.

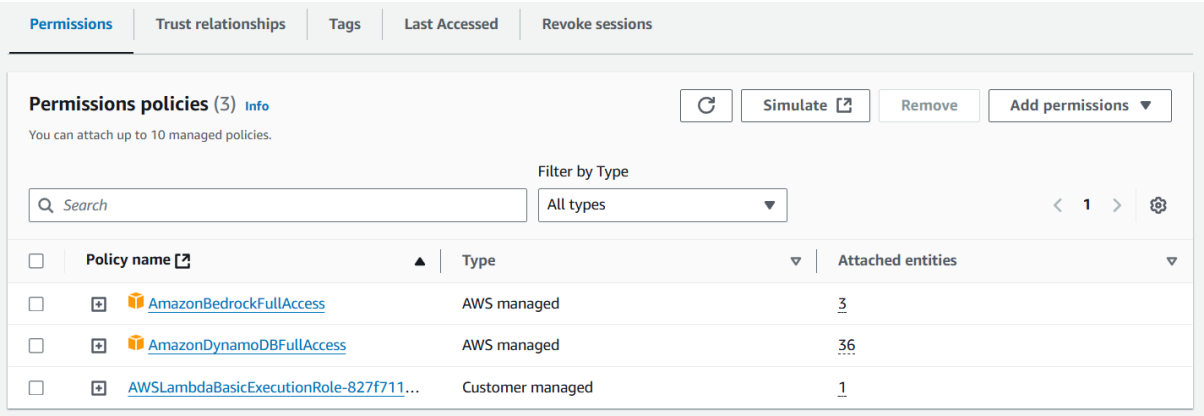


7. Once your function is created you need to go to configuration and click on edit to increase the timeout as you can see below.



8. Then go to Permission and click on the role to edit it with the appropriate permission. So, for the permission you need to add Dynamo DB full access and Bedrock full access.





9. Then use the lambda code file you get with this lab copy the code and then paste it in the lambda function.

```
import json
#1 Import boto3 and create client connection with DynamoDB - Link to documentation - https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/dynamodb/client/get_item.html
import boto3
client=boto3.client('dynamodb')

def lambda_handler(event, context):
#2 Print event value and store the event details in a variable
    print(f"This is the input from agent{event}")
    account_id=event['parameters'][0]['value']#3 Create a request syntax to retrieve data from the DynamoDB Table using GET Item method - https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/dynamodb/client/get_item.html
    response = client.get_item(
        TableName='customerAccountStatus',
        Key={'AccountID':{'N': account_id}})
#4 Store and print the response
    print(response)
#5 Format the response as per the requirement of Bedrock Agent Action Group - https://docs.aws.amazon.com/bedrock/latest/userguide/agents-lambda.html
    response_body = {
        'application/json': {
            'body': json.dumps(response)
        }
    }

    action_response = {
        'actionGroup': event['actionGroup'],
        'apiPath': event['apiPath'],
        'httpMethod': event['httpMethod'],
```

```
    'httpStatusCode': 200,
    'responseBody': response_body
  }

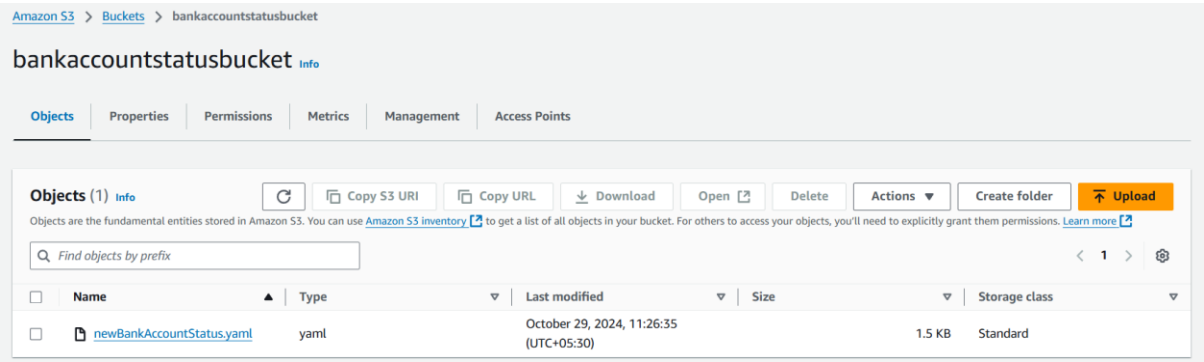
  session_attributes = event['sessionAttributes']
  prompt_session_attributes = event['promptSessionAttributes']

  api_response = {
    'messageVersion': '1.0',
    'response': action_response,
    'sessionAttributes': session_attributes,
    'promptSessionAttributes': prompt_session_attributes
  }

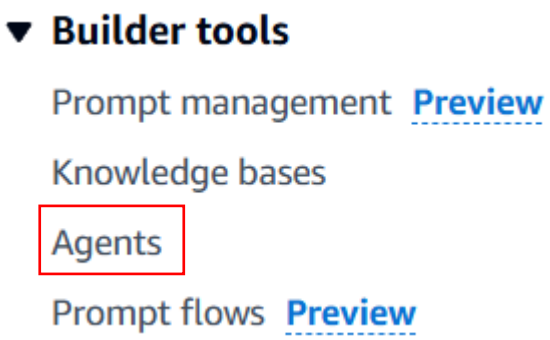
  return api_response

#6 Print the final response
  # TODO implement
```

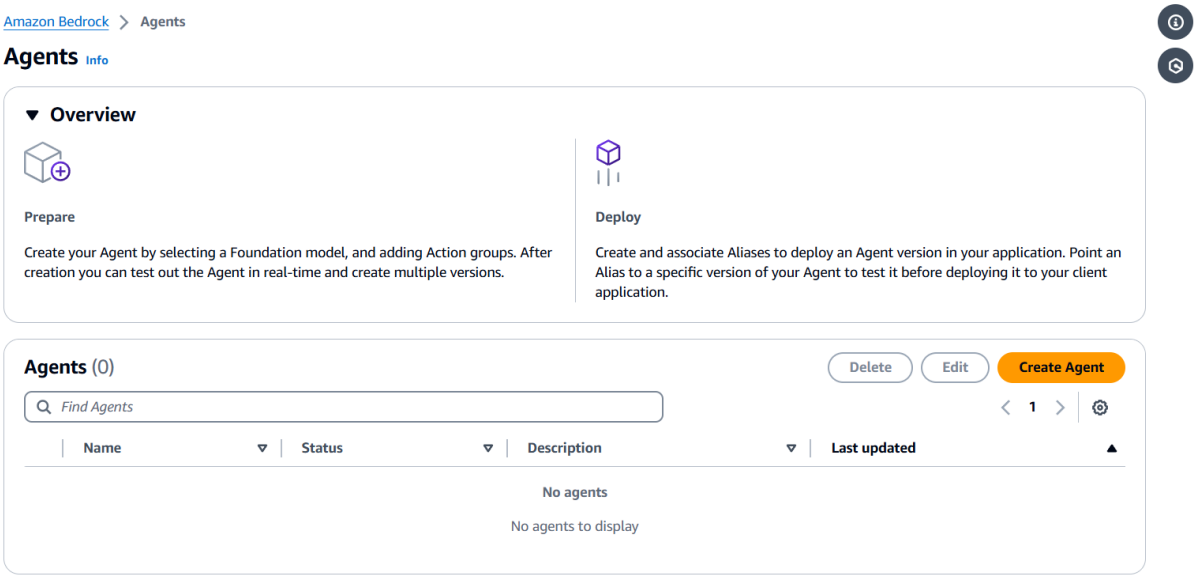
10. The Bedrock agent uses an OpenAPI schema to determine which Lambda function to invoke based on user queries. The schema includes details like API title, version, and descriptions to map user inputs, such as account ID. It specifies the request parameters and response formats for the Lambda function. This structured approach allows for clear communication between the Bedrock agent and the Lambda functions. Finally, the schema must be uploaded to an S3 bucket for the Bedrock agent to access it during operation.
11. So, you need to create an S3 bucket and upload the YAML to it. You will get this YAML with the lab. Also, read the YAML file.



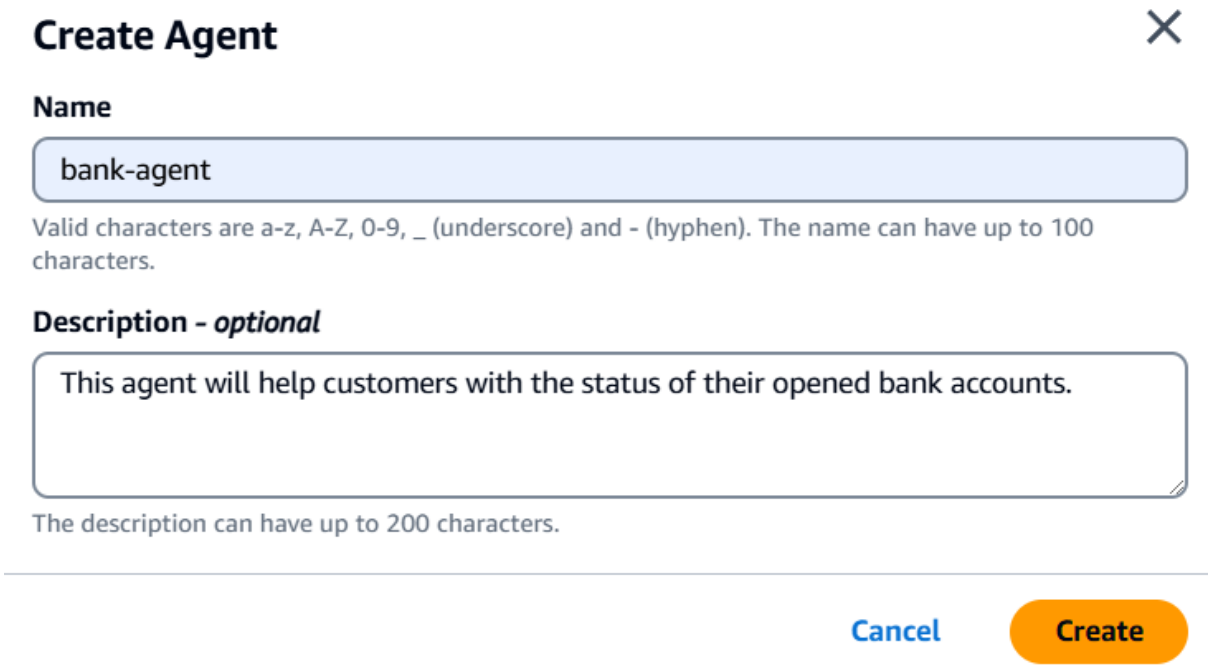
12. After that we are going to create the Bedrock agent for that first navigate to Bedrock. Here you need to expand Builder tools and click to open Agents.



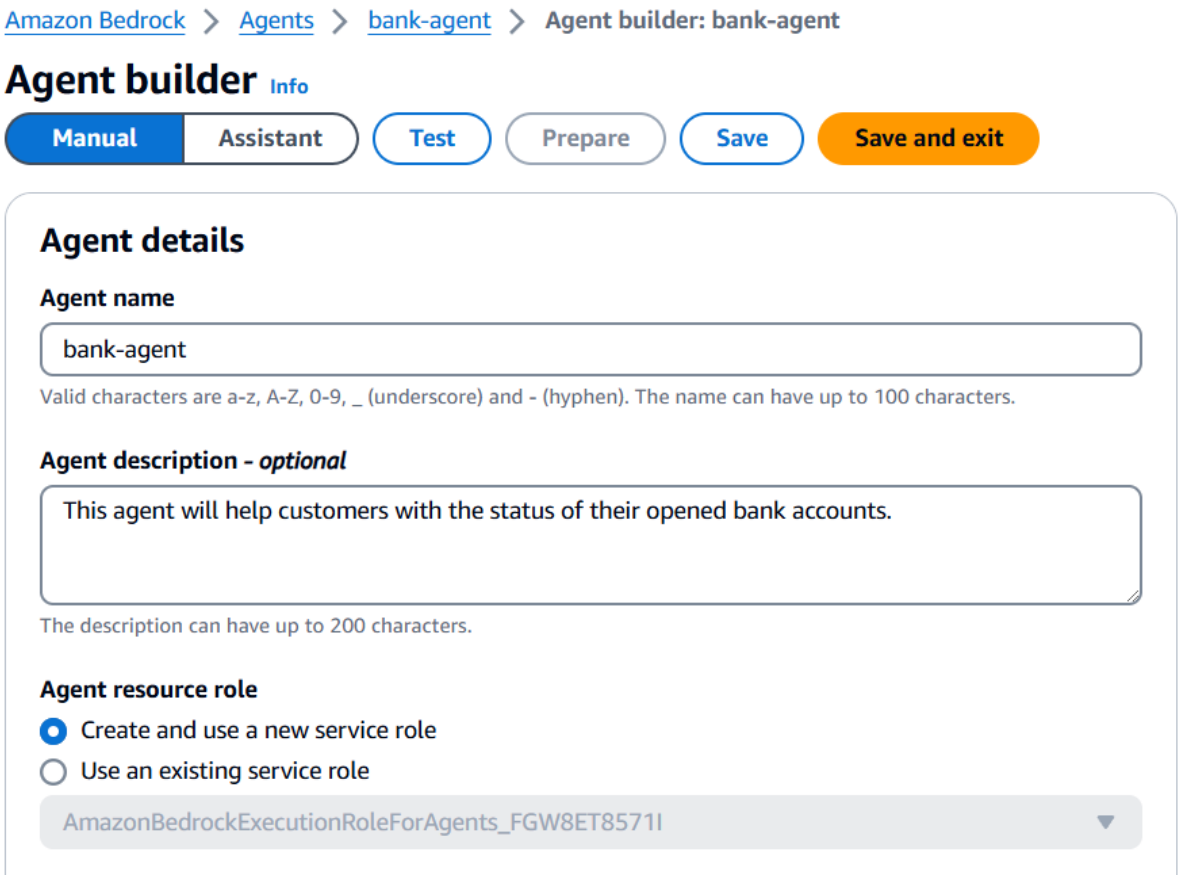
13. First, we are going to create our agent. Click on create agent.



14. First, give this agent a name and a description and click on create.

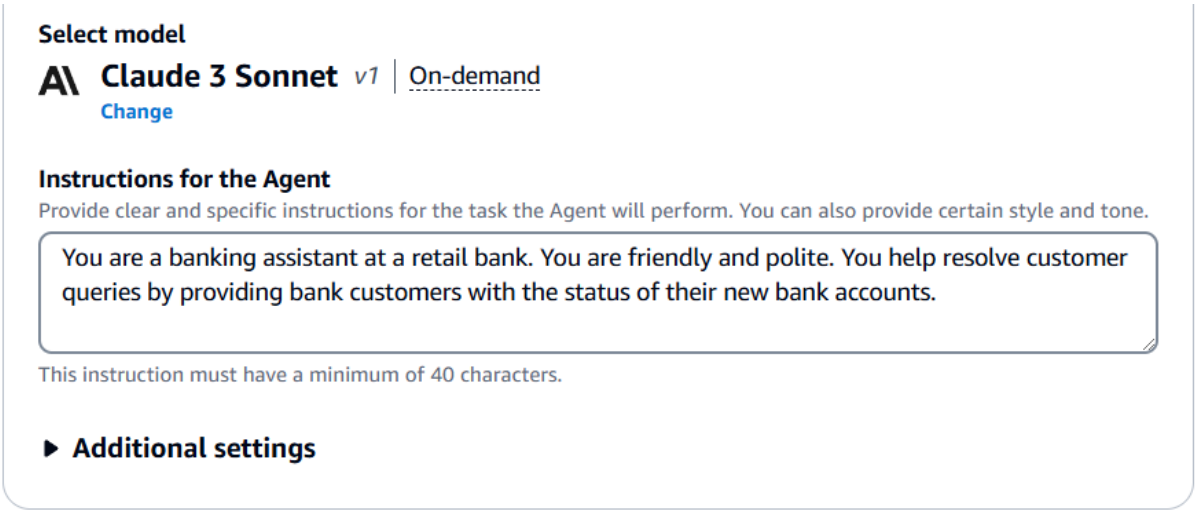


15. Now you need to choose various properties for your agent so, here you can see that we need to create a new role for our agent.

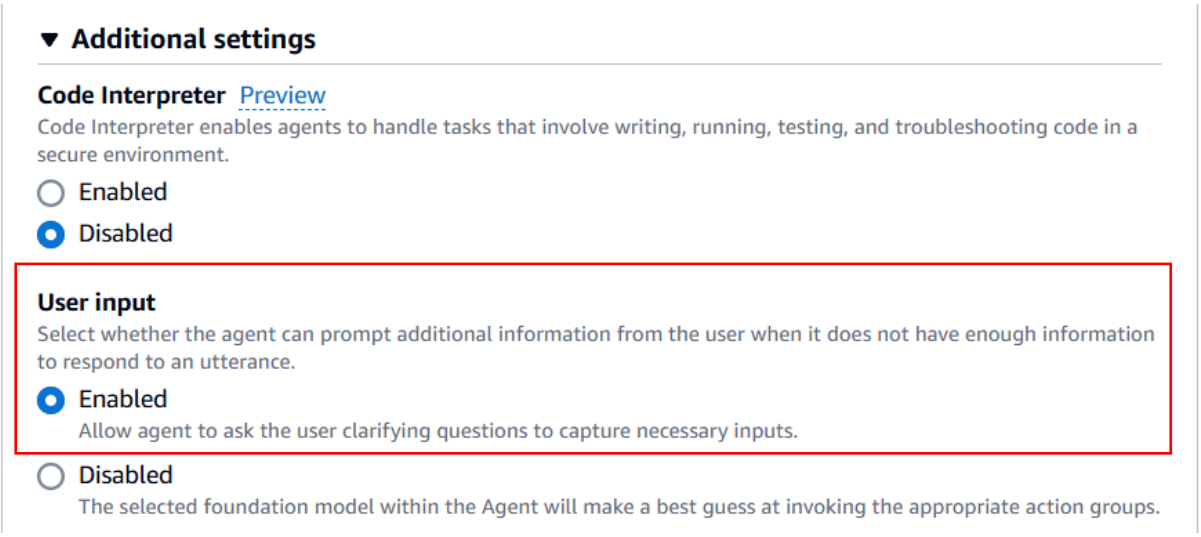


16. Then we are selecting Claude 3 Sonnet as our model and we are proving an instruction for our agent. If this model does not work or it gives errors then you can change it.

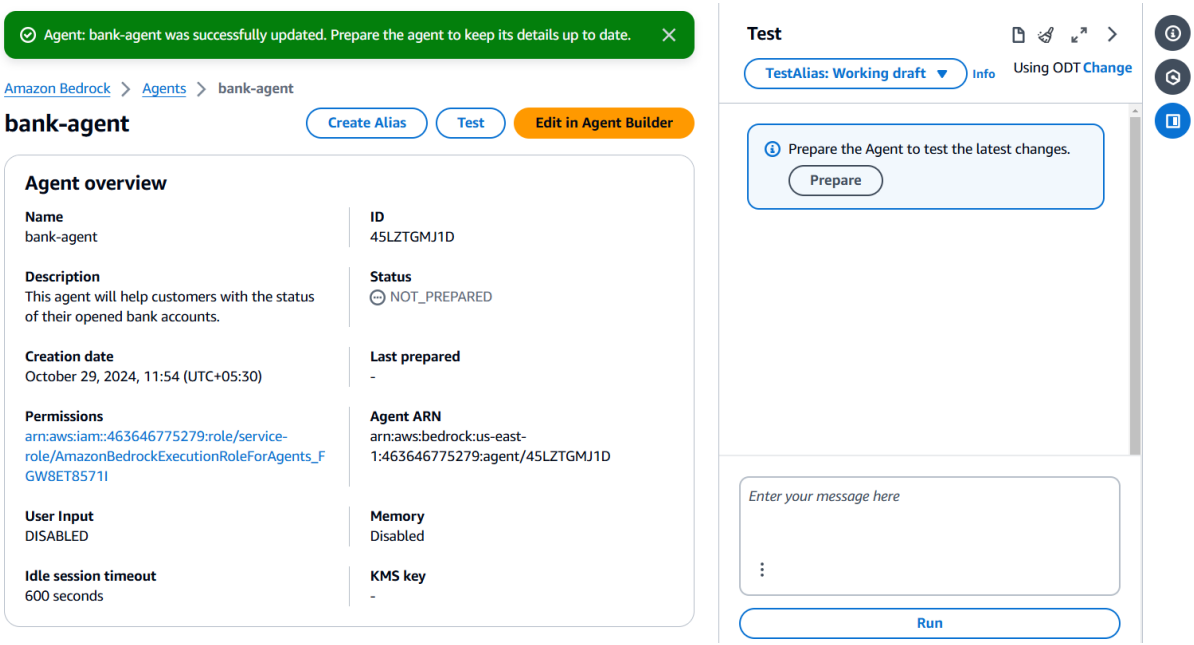
    "You are a banking assistant at a retail bank. You are friendly and polite. You help resolve customer queries by providing bank customers with the status of their new bank accounts."



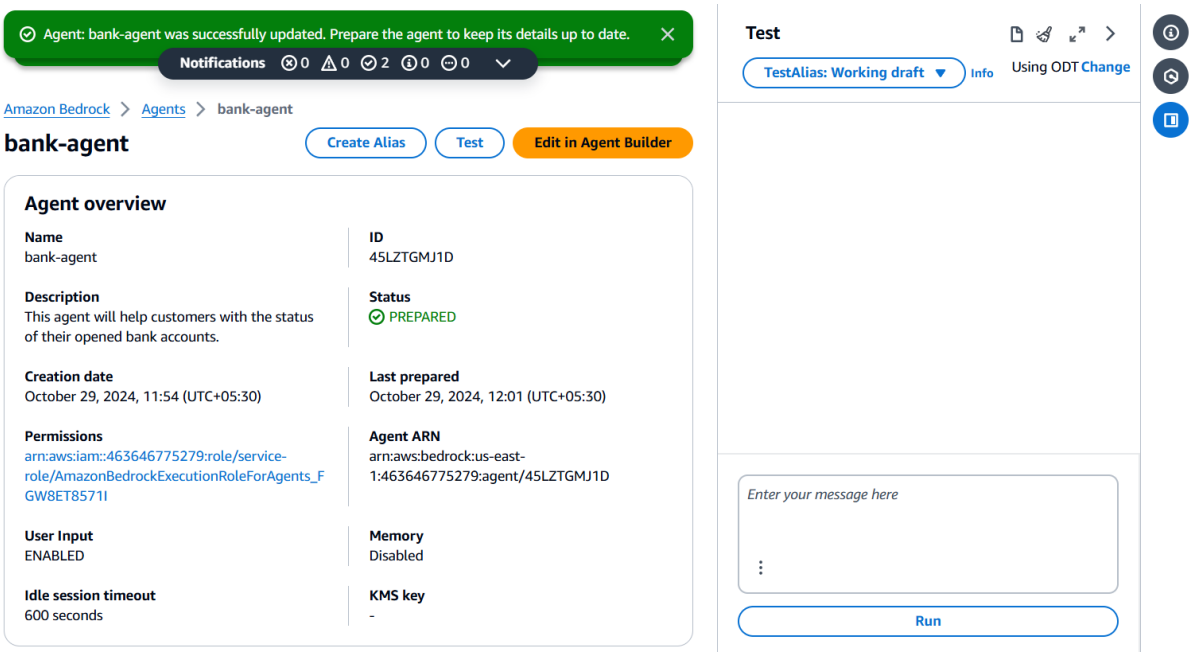
17. Expand the additional settings and enable the user input.



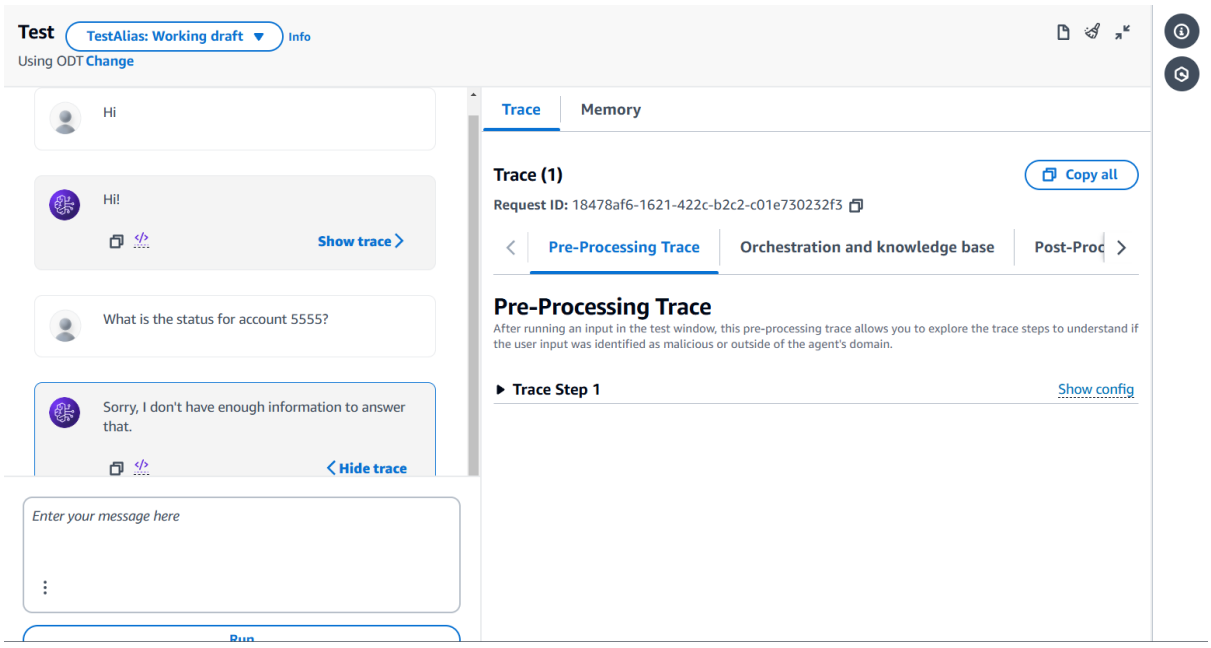
18. After that scroll up and click on save and click on save and exit button to save your changes. Below you can see that all the changes were saved now we need to click on Prepare to prepare the agent.



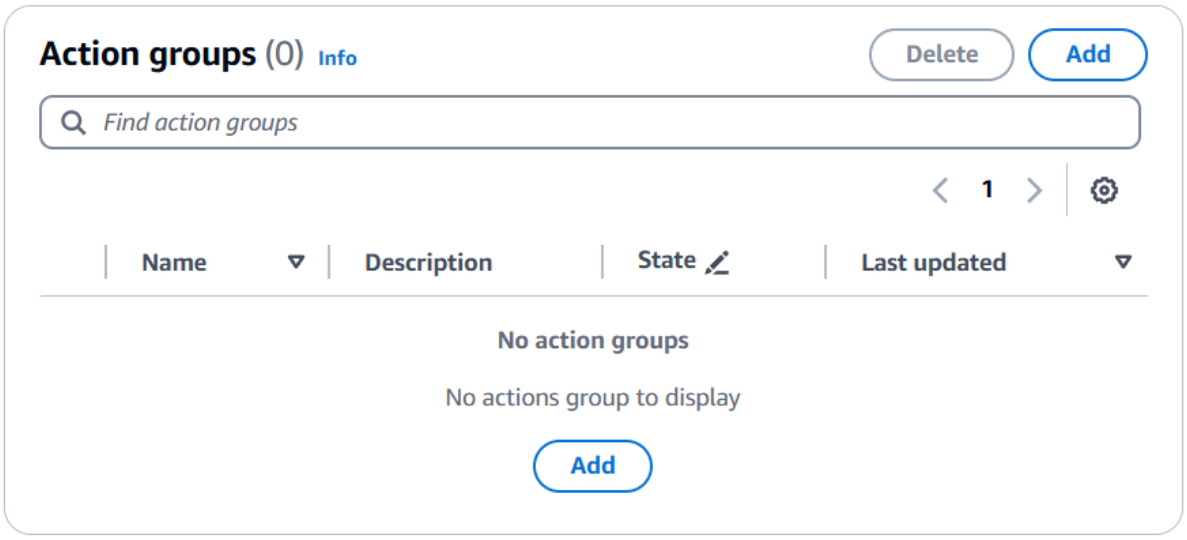
19. Here you can see that our agent status has been changed to prepared.



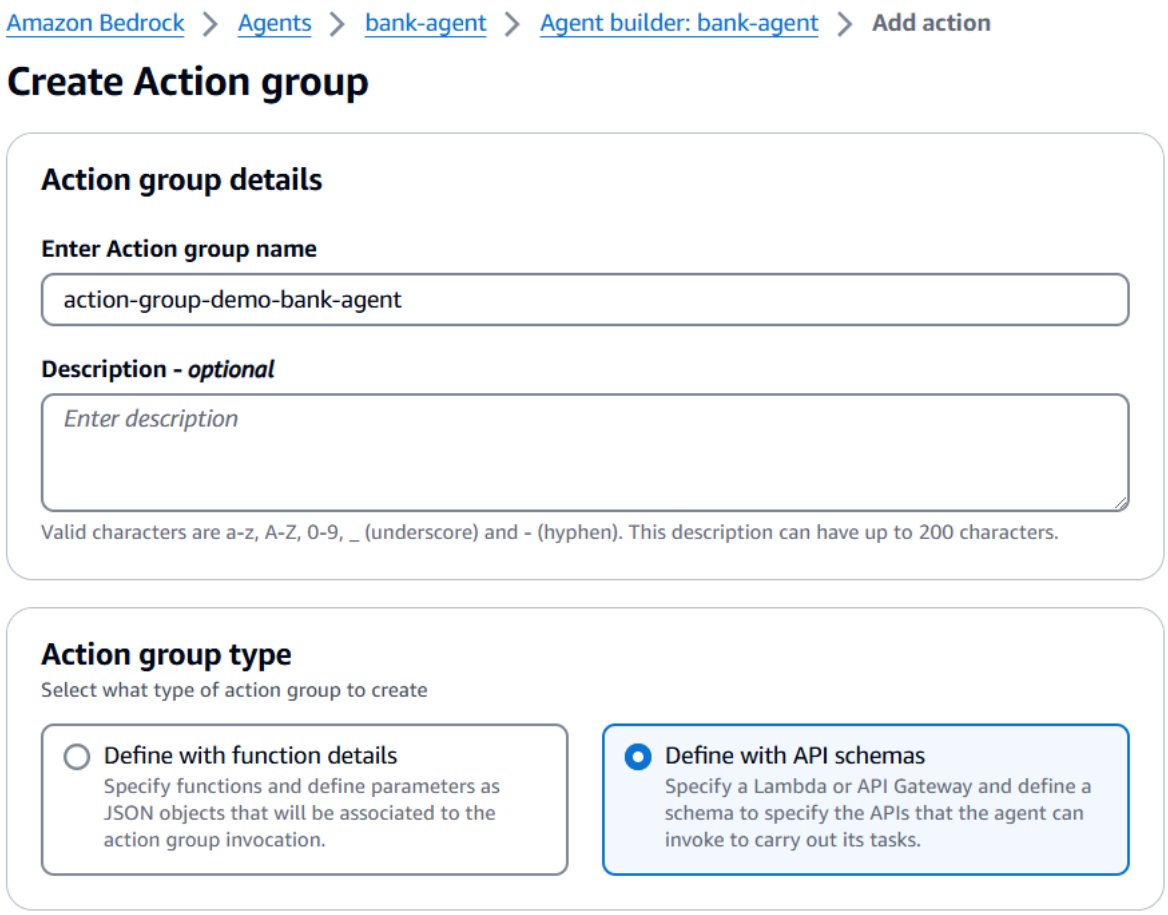
20. Here you can see that our model is responding us but it does not have the information for the accounts.
21. For that we need to create an action group for our agent. So, you need to click on Edit in the agent builder button.



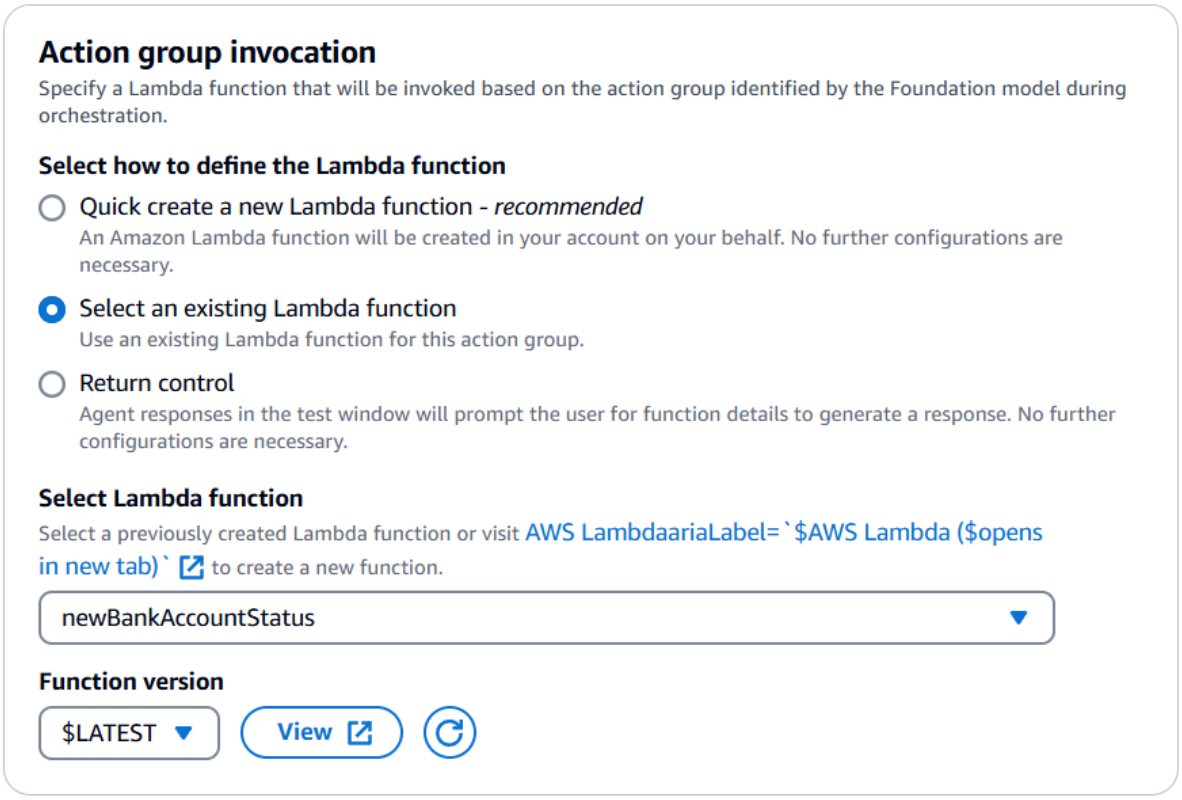
22. Then scroll down to action groups and click on Add.



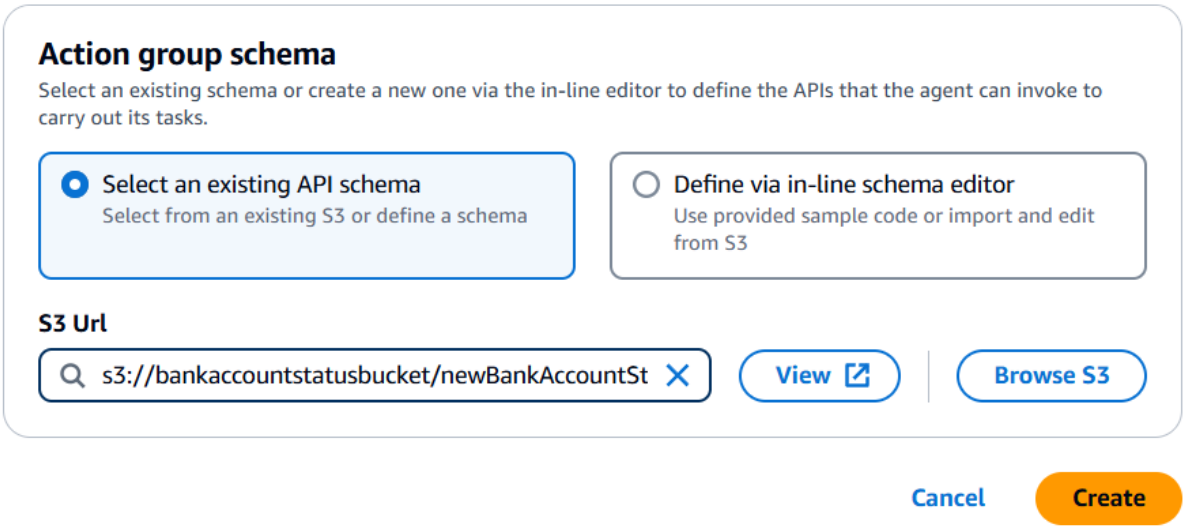
23. First, you need to give it a name then in the action group type choose Define with API schemas.



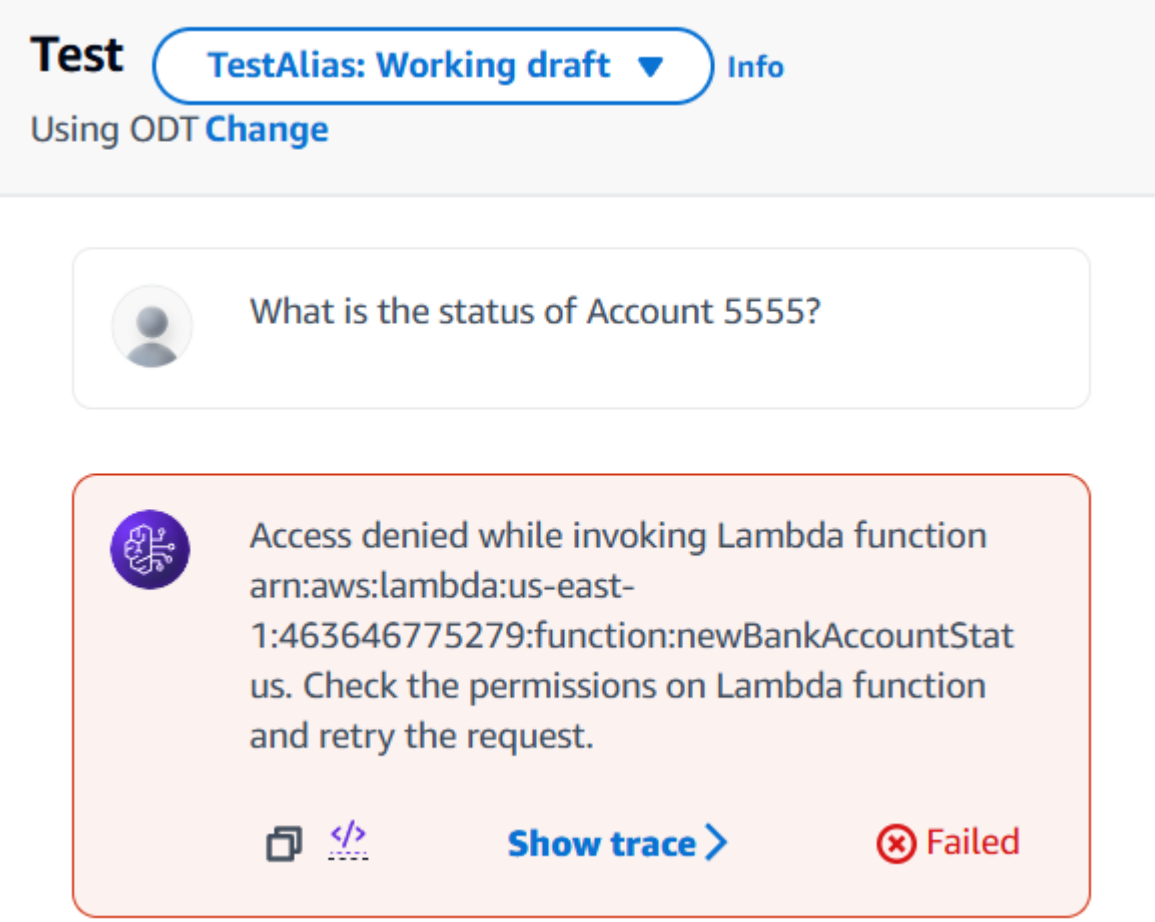
24. Here you need to click on Select an existing Lambda function and then choose your lambda function.



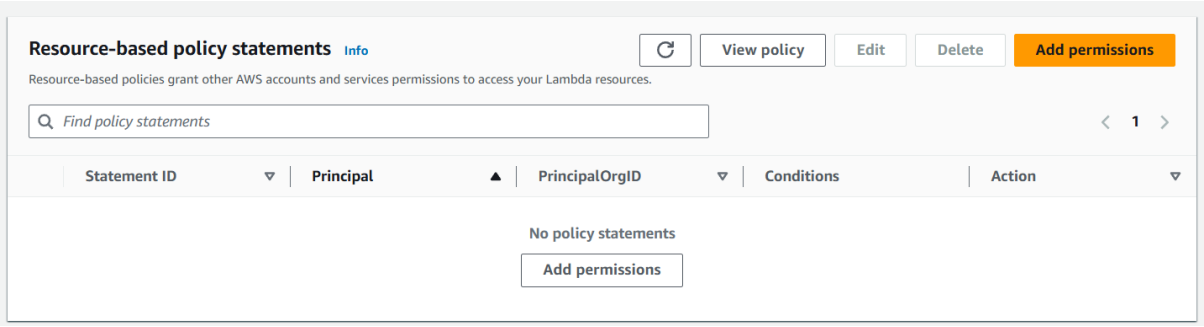
25. After that in the action group schema you need click on select an existing API schema and give the URL of your YAML file stored in the S3 bucket. Then click on Create.



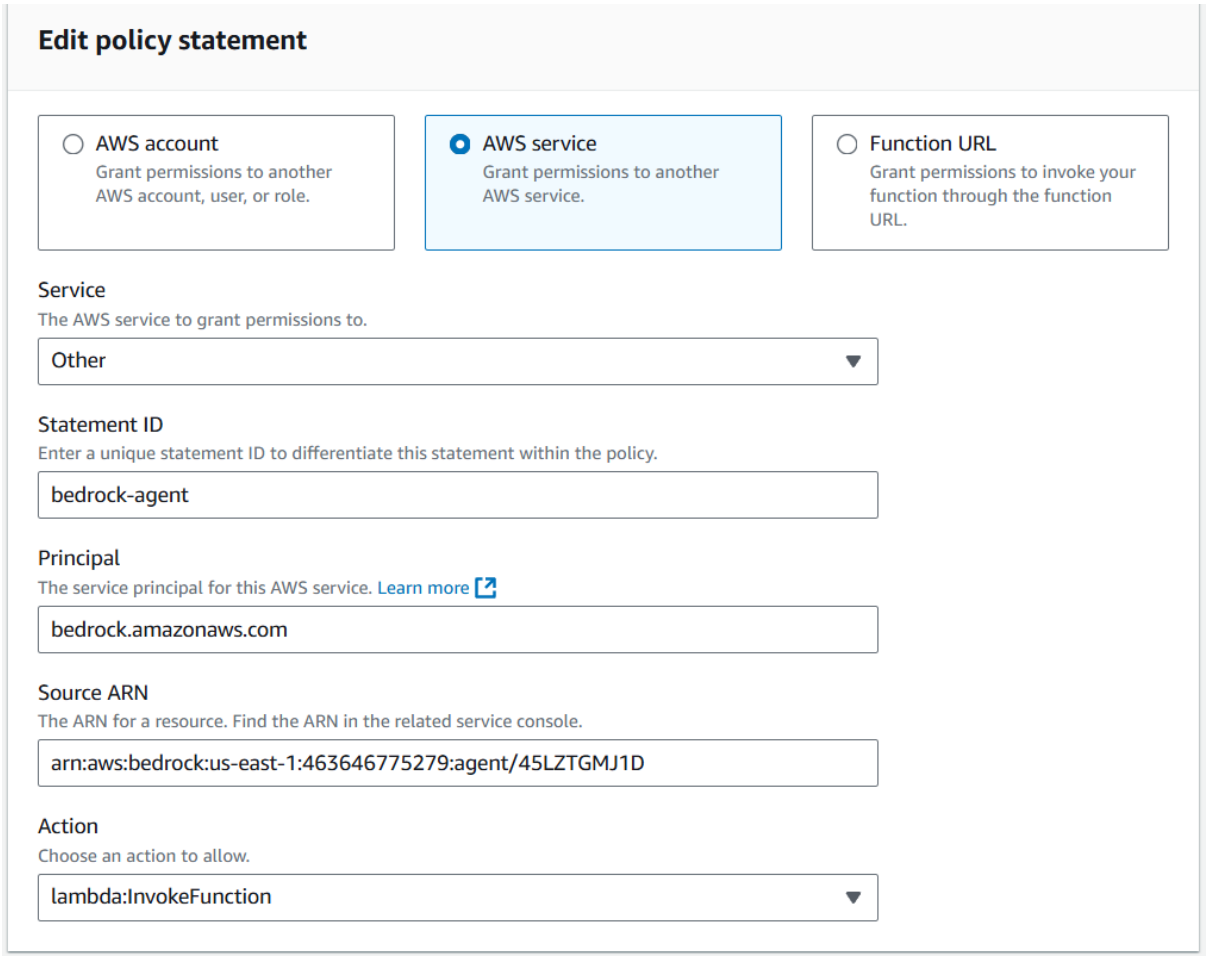
26. Once your action group has been created then you need to click on save and exit. Also click on prepare to prepare your data for the new settings.
27. Now if you ask this agent the questions about the account status then you will see that it is still giving you the errors.



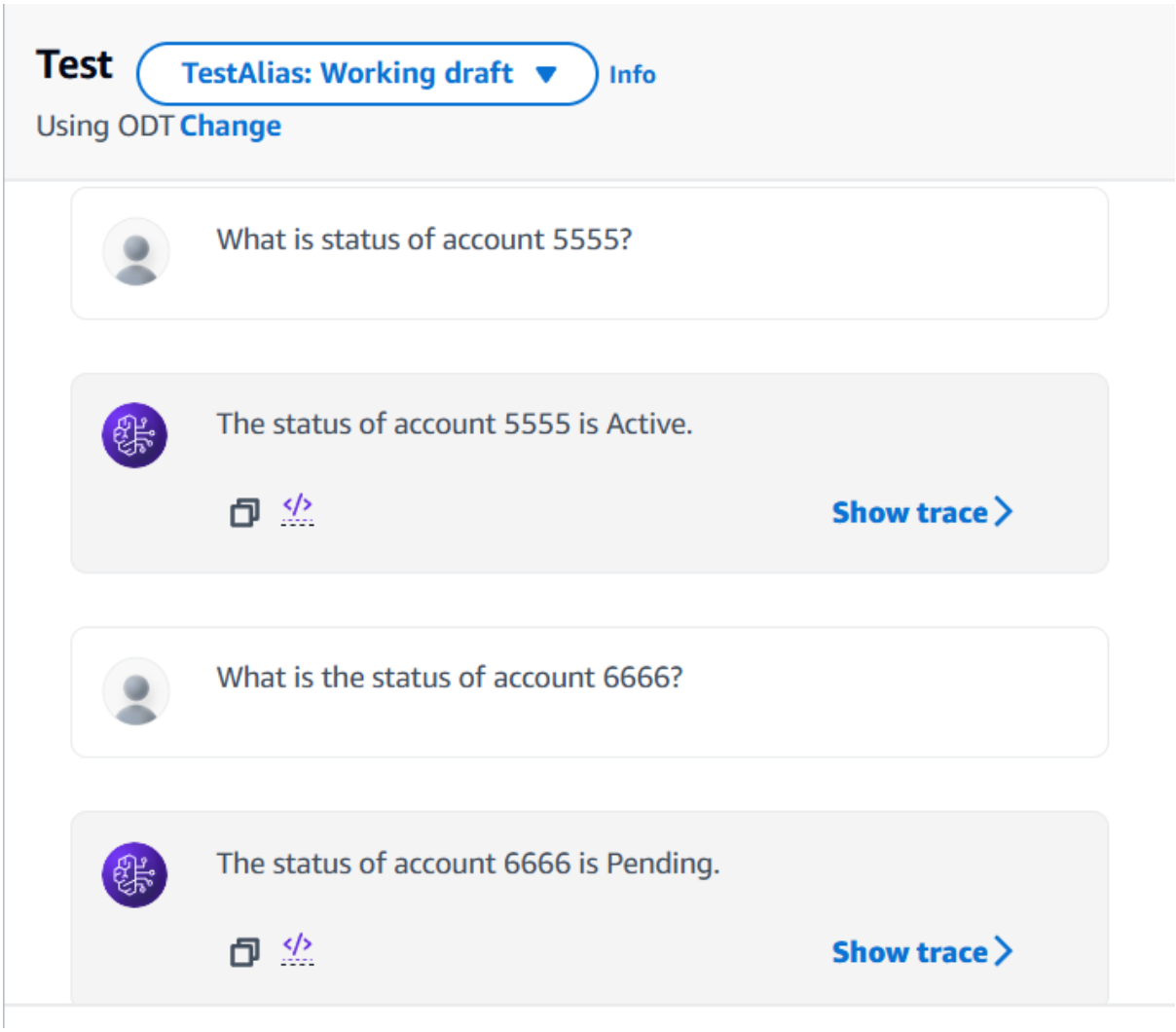
28. So, to resolve it you need to come back to your lambda function. Go to configuration, then to permission and scroll down to resource-based policy statements. Click on add permission.



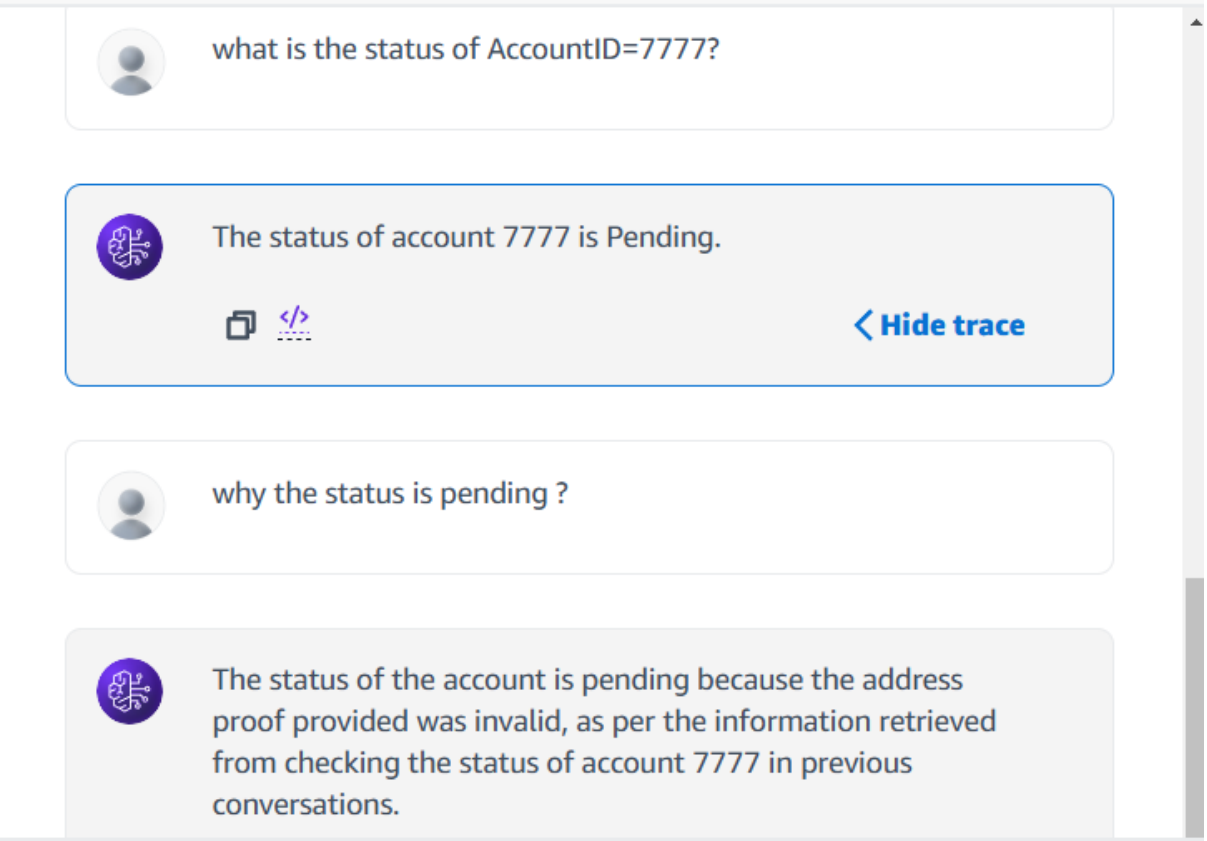
29. Here you need to choose AWS Service and, in the service, choose others. Then int eh Statement ID, Principal write the same as you can see below.
30. For the source ARN you need to copy the ARN of your Bedrock agent and paste here. In the action you need to choose Lambda invoke function, same as shown below. Then click on save.



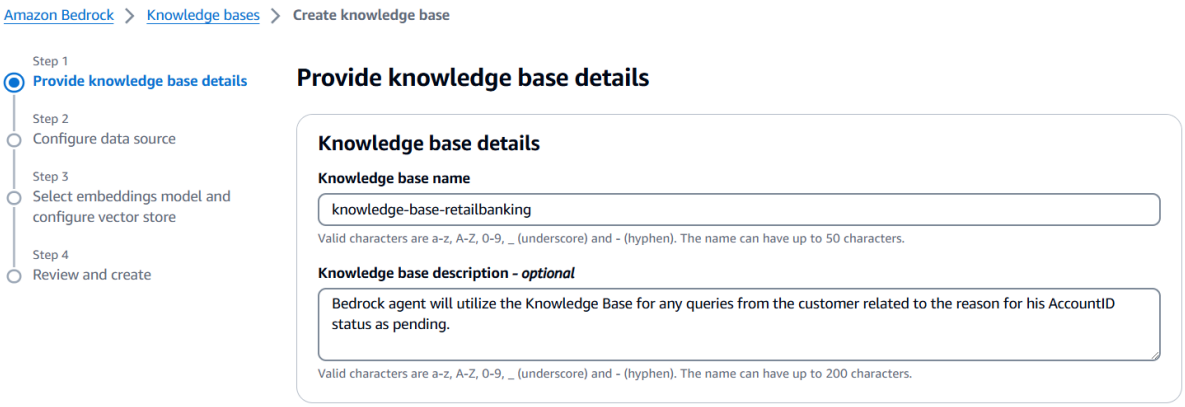
31. Now you can see that is has answered us the question for status of bank accounts.



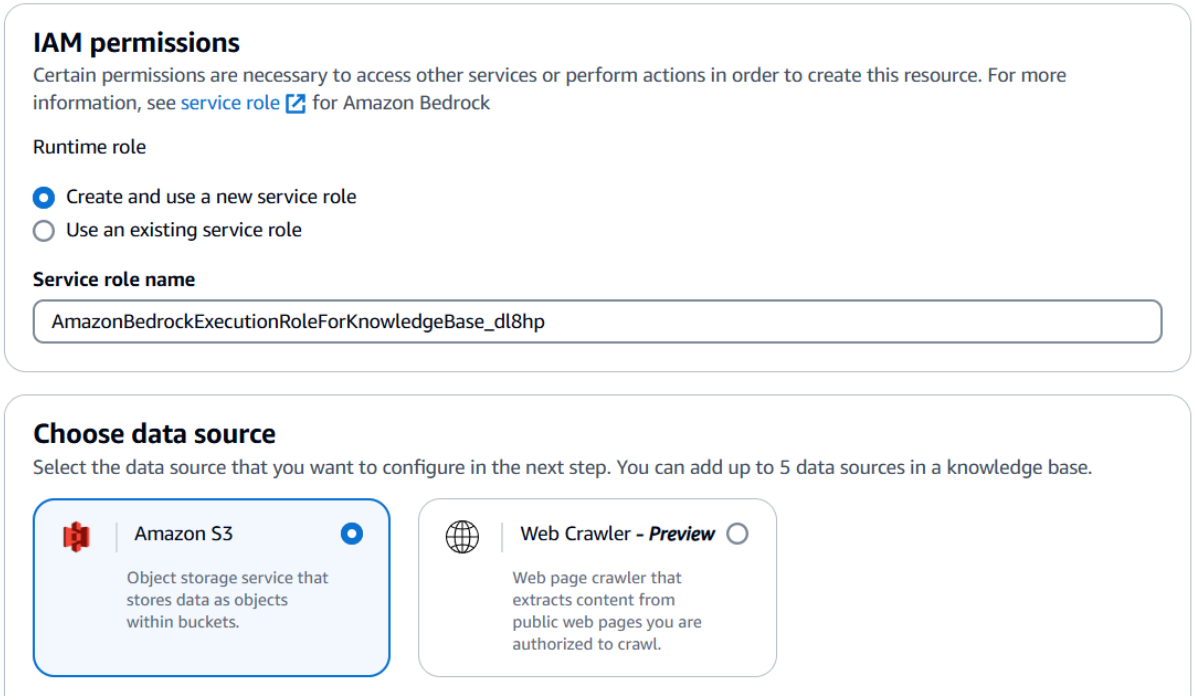
32. It will also answer you if you ask it the proper questions too.



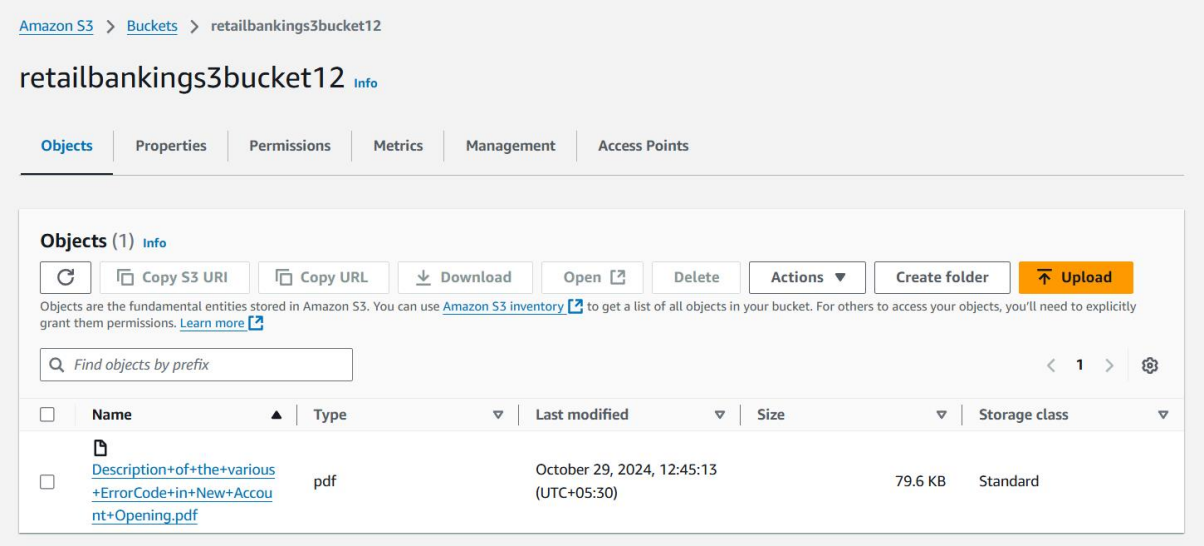
33. The Bedrock agent can handle user queries about error codes like "invalid identification" by integrating with a Retrieval-Augmented Generation (RAG) approach. Users can ask for explanations of these codes, which aren't fully covered in DynamoDB. A PDF document detailing these codes is created and uploaded to an S3 bucket. The RAG solution breaks this PDF into chunks and generates vector embeddings for storage. When a user poses a question, it retrieves relevant information from the vector store, providing comprehensive answers.
34. First, we are going to create knowledge base in Bedrock. Remember you cannot create it using your root account you need to create an IAM account for it.
35. Now in your bedrock service go to Knowledge base and click on create. In step 1, you need to give it a name and description.

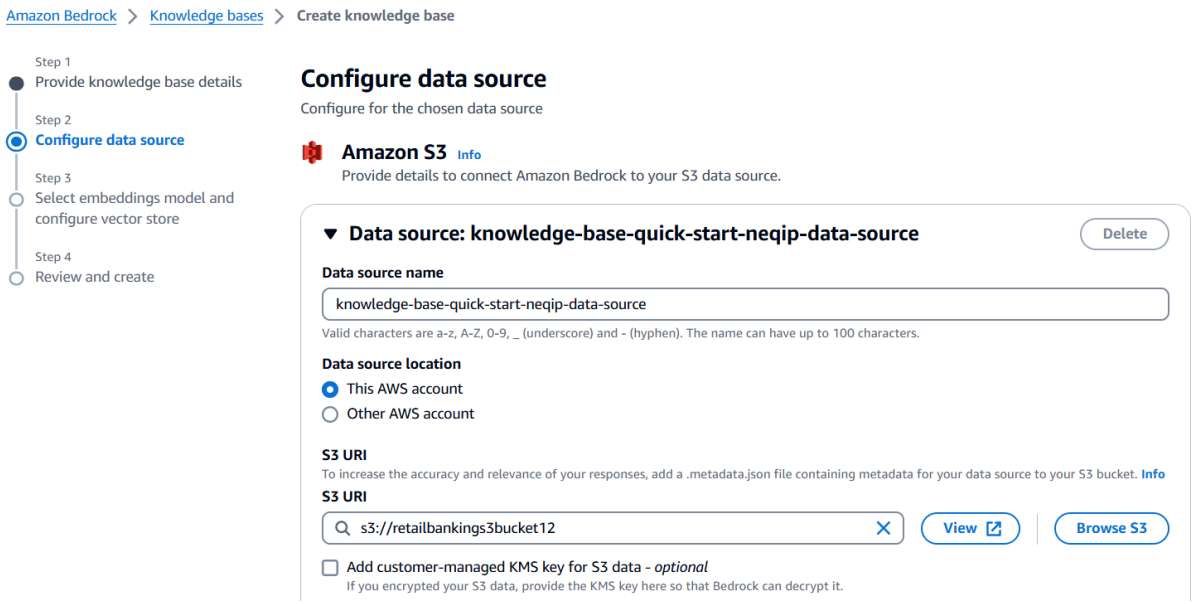


36. Then for the IAM permissions choose to create new role and for the data source choose S3 bucket. Click on next.

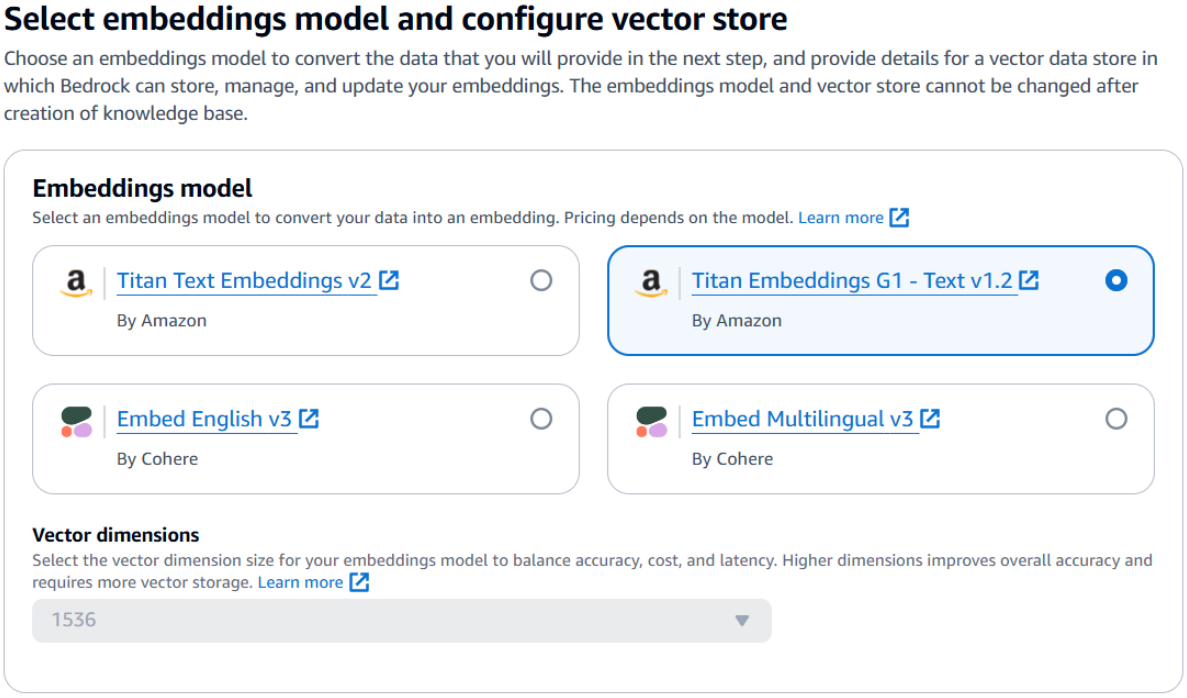


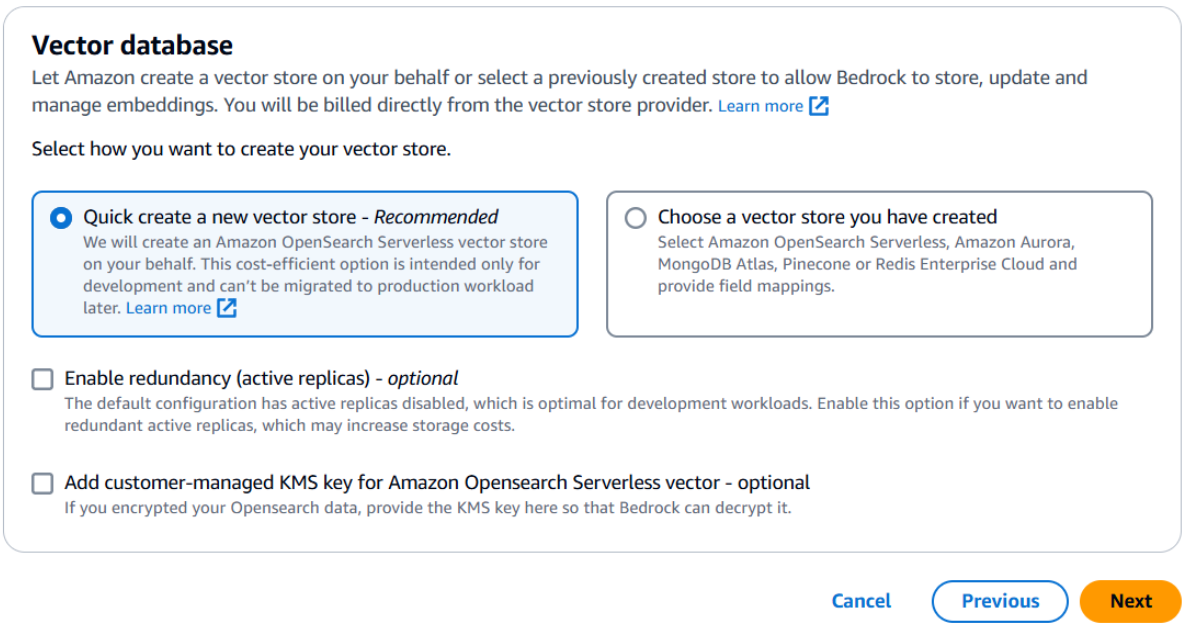
37. In the step 2, you need to configure your S3 bucket. But first you need to create a new S3 bucket and upload the PDF document to it. Then come back to bedrock and choose your S3 bucket.



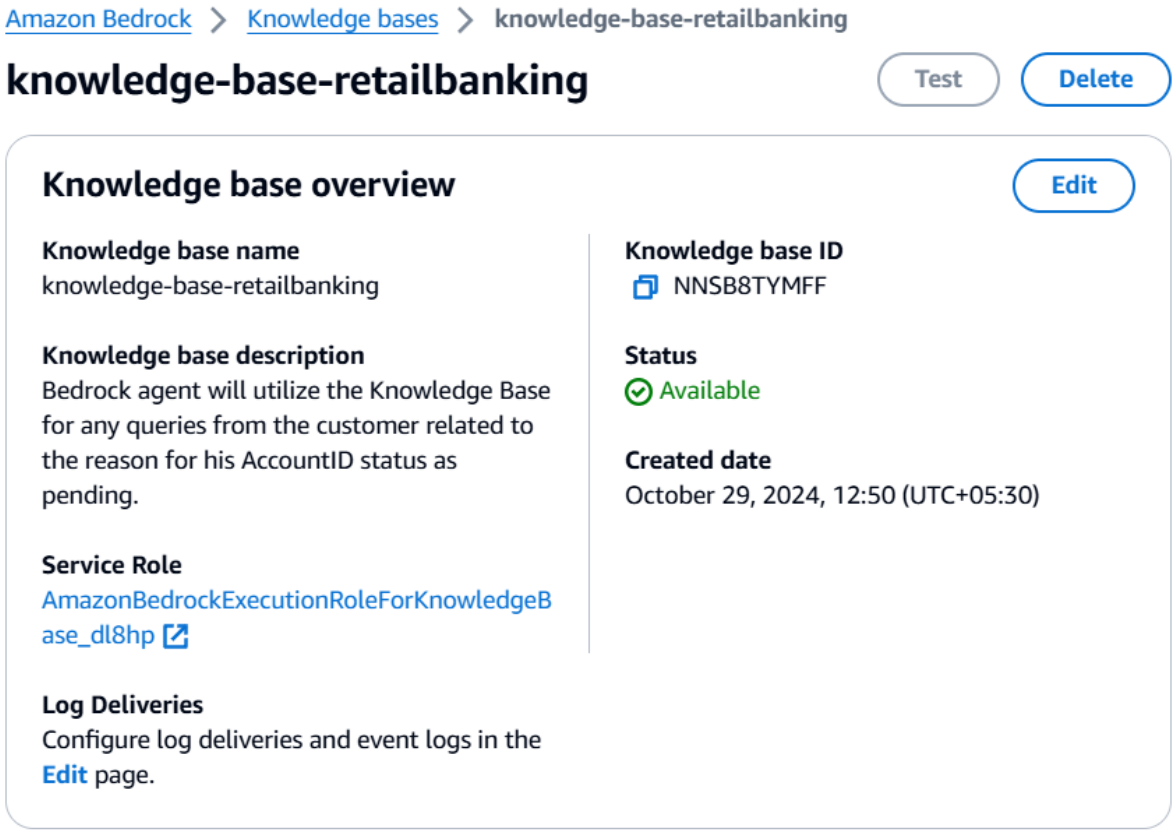


38. In step 3, choose Titan embeddings G1 as your embedding model, and for vector base choose Quick create and click on next.
39. Then from the review page you need to click on create knowledge base.

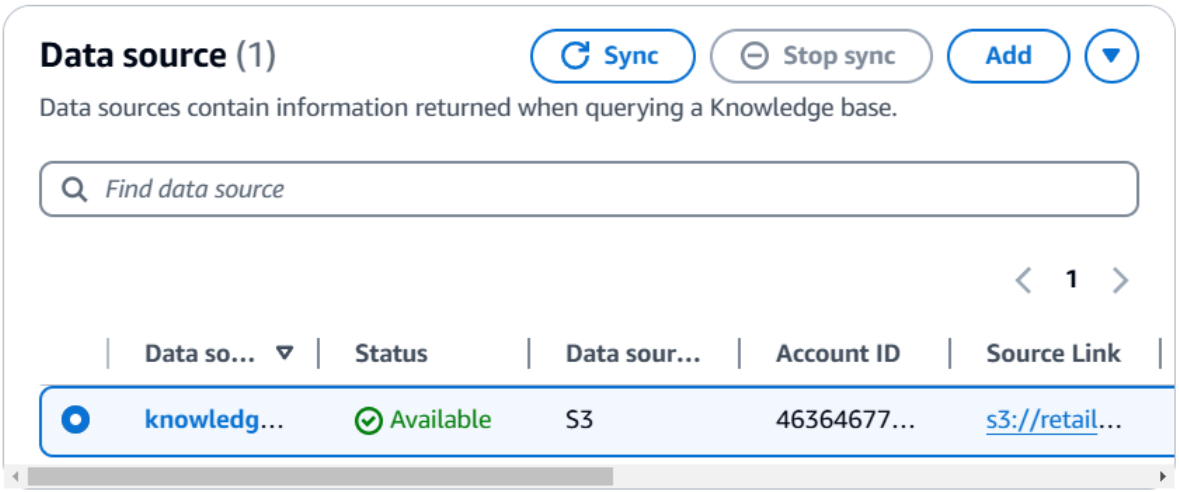




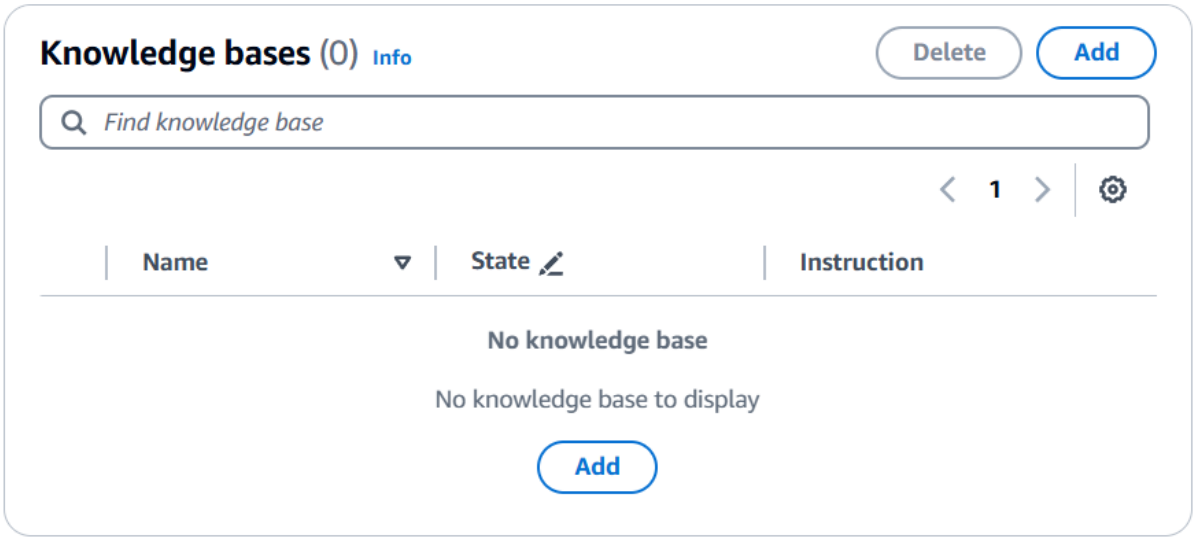
40. The creation of a knowledge base will take 4-5 minutes may be more. Below you can see that our knowledge base has been created.



41. Now scroll down to the data source section and select your data source then click on sync.

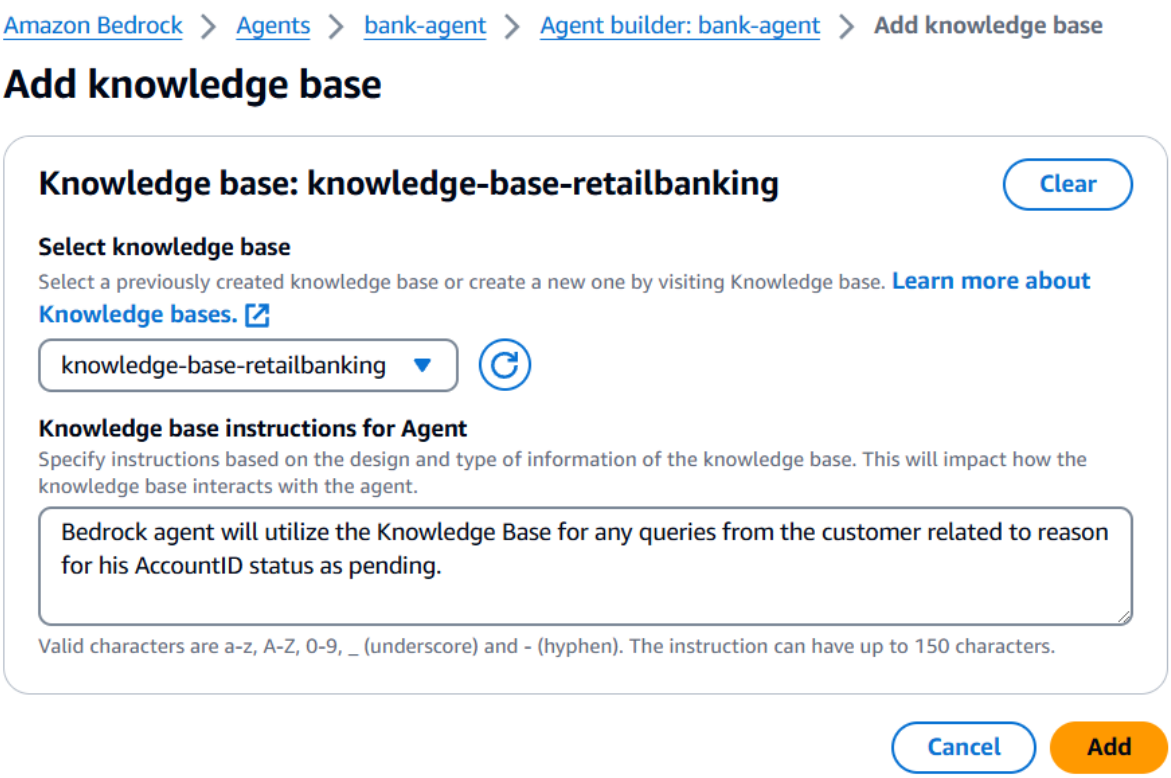


42. Once your knowledge is ready go back to your agent and click on edit in agent builder. After that scroll down to knowledge bases and click on Add.

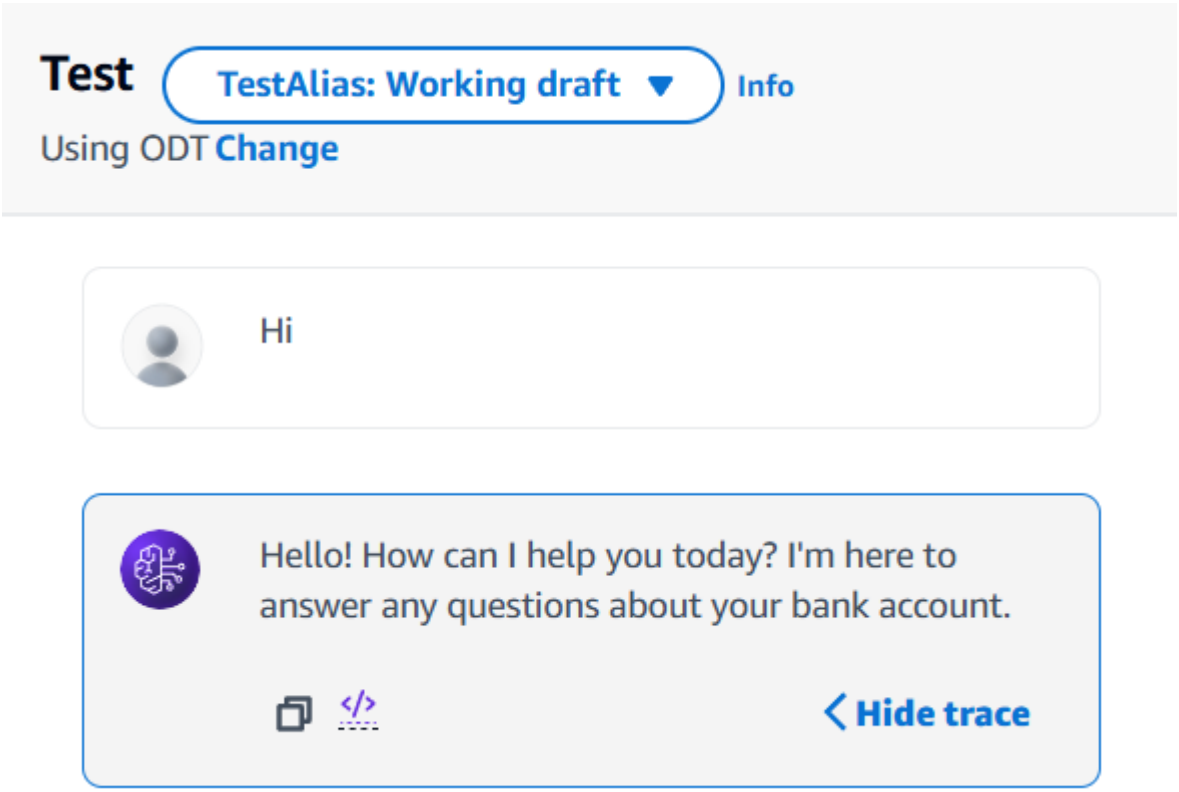


43. Then you need to choose your knowledge base and give it an instruction. Click on save exit.

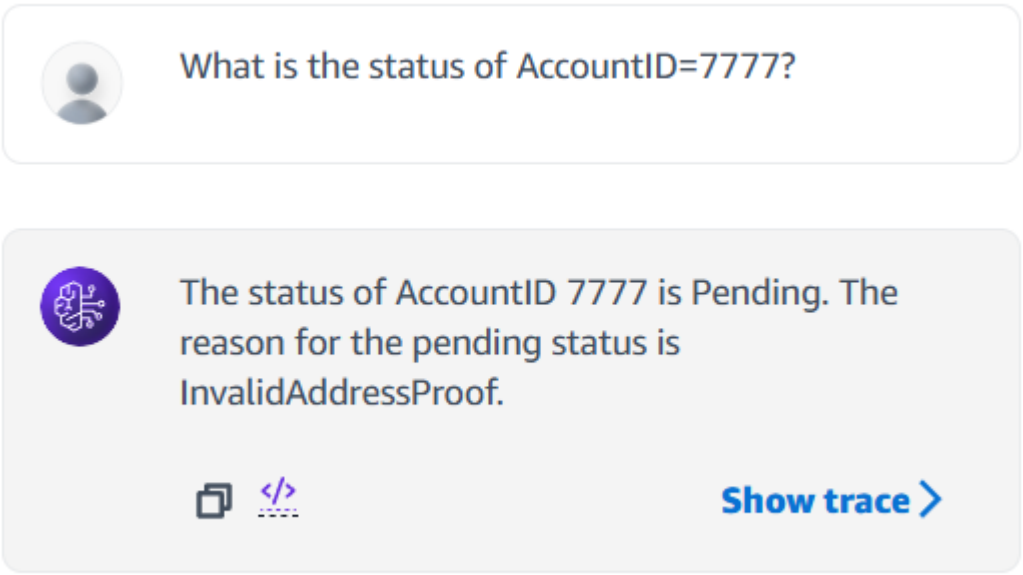
    "Bedrock agent will utilize the Knowledge Base for any queries from the customer related to reason for his AccountID status as pending."



44. After that prepare your data. Here you can see that we just greeted our agent and it responded back properly this time.



45. After that we asked it another question and we got the answer for that too.



46. Now delete all the resources, start by deleting the Knowledge base then the agent and rest of the things.